

Software Engineering

Team Programming Assignment

For this project, we used C and C++. We used Microsoft unit testing using the native test code. We used the native test project to create different tests for our project. As a team we used two IDEs for our project. Two of our team members (Petros and Raman) used code blocks where as the other three of us used the Visual Studio. Most of our team members used the Windows operating system except Petros who used Linux (KDE) for the project.

Throughout this project, we first learnt how to divide the work among our team members through the Work Break Down System (WBS) and Gantt Project. This project required us to work closely together. We learned how to make the project more systematic and user friendly through testing and by implementing the evolution process. Thus we learned how to use different testing and evolution criteria to make the game more user-friendly and less error prone.

This project helps prepare us for the real world. In the industry, these sort of procedures and team involvement are preferred. Through this project we learned the benefits of working in a group, and figured out the part each team member had made. This industrial process of working on a big project ensures that no one is lagging behind on his/her allocated work and this project made us all able to communicate with each other. These things are important in the industry to succeed, which is why this assignment helped us understand the connection of different topics of our course text book to real world.

In this assignment, we find coordination to be a tough task. It was difficult to combine the work done by each member. The tough part was to build a randomized map, to find/come up with algorithms for that, finding/coming up with pathfinding algorithms, as well as the testing part. As a team, we found creating the commands to be easy and available. For this assignment, we came up with two different custom randomization algorithms to make our maze. We also implemented custom algorithms for the shortest path finding.

Dependable programming:

- (i) Limit the Visibility of information in a program: We limited the visibility of data in the source code using modularization, and hid any debug information from the users unless required in the program.
- (ii) Check all the inputs for validity: The game will accept the input from the user. It will ask for a command, and also for an optional number of steps. The command I/O structure by itself requires us to check the user's input validity, making sure that the user provided the proper characters/strings in the command and the proper/correct integer value for the number of steps.

- (iii) Validity Checks: Our program checks the range of the numbers the user typed. If the user provides any numbers other than from 2 to 9, the program will not accept the input.
- (iv) Minimize the use of error-prone constructs: We have used the minimum error-prone constructs. We will use pointers in our program, but we will also make sure they do not create any ambiguity during the reference.
- (v) Check array bounds: We have properly checked for array bounds during development, and no new arrays will be created in the game. All the loops used are ensured to be within bounds of any handled array.
- (vi) Include timeouts when calling external components: We do not have any notion of timeouts in our program since there will be no calling of external components.